

STYLECART-FASHION E-COMMERCE DATABASE

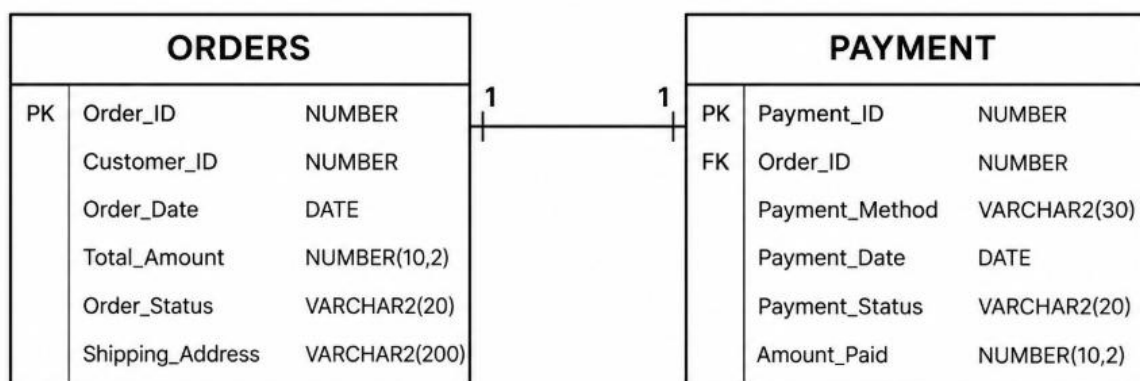
PAYMENT MANAGEMENT SYSTEM

1. Database Design

1.1 Payment Table

Attribute	Data Type	Constraint
Payment_ID	NUMBER	PRIMARY KEY
Order_ID	NUMBER	FOREIGN KEY
Payment_Method	VARCHAR2(30)	NOT NULL
Payment_Date	DATE	NOT NULL
Payment_Status	VARCHAR2(20)	NOT NULL
Amount_Paid	NUMBER(10,2)	NOT NULL

2. Table Relationships



The Payment table is related to the Orders table using Order_ID.

- Orders table: Order_ID is the Primary Key (PK).
- Payment table: Order_ID is the Foreign Key (FK).
- The foreign key connects each payment record to its related order.
- In our design, the relationship is 1 : 1 — one order has one payment record.

3. Table Creation

3.1 Create Payment Table

```
SQL> CREATE TABLE Payment (  
  2     Payment_ID NUMBER PRIMARY KEY,  
  3     Order_ID NUMBER NOT NULL,  
  4     Payment_Method VARCHAR2(30) NOT NULL,  
  5     Payment_Date DATE NOT NULL,  
  6     Payment_Status VARCHAR2(20) NOT NULL,  
  7     Amount_Paid NUMBER(10,2) NOT NULL,  
  8     FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)  
  9 );
```

Table created.

4. Payment Data Insertion

```
SQL> INSERT INTO Payment  
  2 (Payment_ID, Order_ID, Payment_Method, Payment_Date, Payment_Status, Amount_Paid)  
  3 VALUES  
  4 (501, 1001, 'UPI', SYSDATE, 'Successful', 2098.00);
```

1 row created.

```
SQL>  
SQL> INSERT INTO Payment  
  2 (Payment_ID, Order_ID, Payment_Method, Payment_Date, Payment_Status, Amount_Paid)  
  3 VALUES  
  4 (502, 1002, 'Credit Card', SYSDATE, 'Successful', 1599.00);
```

1 row created.

```
SQL>  
SQL> INSERT INTO Payment  
  2 (Payment_ID, Order_ID, Payment_Method, Payment_Date, Payment_Status, Amount_Paid)  
  3 VALUES  
  4 (503, 1003, 'Debit Card', SYSDATE, 'Failed', 899.00);
```

1 row created.

```
SQL>  
SQL> COMMIT;
```

Commit complete.

5. Manage Payment Transactions

```
SQL> SELECT * FROM Payment;
```

PAYMENT_ID	ORDER_ID	PAYMENT_METHOD	PAYMENT_D
PAYMENT_STATUS	AMOUNT_PAID		
501	1001	UPI	29-SEP-26
Successful		2098	
502	1002	Credit Card	29-SEP-26
Successful		1599	
503	1003	Debit Card	29-SEP-26
Failed		899	

6. Display Successful Payments

```
SQL> SELECT *
2 FROM Payment
3 WHERE Payment_Status = 'Successful';
```

PAYMENT_ID	ORDER_ID	PAYMENT_METHOD	PAYMENT_D
PAYMENT_STATUS	AMOUNT_PAID		
501	1001	UPI	29-SEP-26
Successful		2098	
502	1002	Credit Card	29-SEP-26
Successful		1599	

7. Display Failed Payments

```
SQL> SELECT *
2 FROM Payment
3 WHERE Payment_Status = 'Failed';
```

PAYMENT_ID	ORDER_ID	PAYMENT_METHOD	PAYMENT_D
PAYMENT_STATUS	AMOUNT_PAID		
503	1003	Debit Card	29-SEP-26
Failed		899	

8. Modify Payment Status

```
SQL> UPDATE Payment
  2 SET Payment_Status = 'Successful'
  3 WHERE Payment_ID = 503;
```

1 row updated.

```
SQL>
```

```
SQL> COMMIT;
```

Commit complete.

```
SQL>
```

```
SQL> SELECT *
  2 FROM Payment
  3 WHERE Payment_ID = 503;
```

PAYMENT_ID	ORDER_ID	PAYMENT_METHOD	PAYMENT_D
503	1003	Debit Card	29-SEP-26
Successful		899	

9. Analyze Payment Methods

```
SQL> SELECT
  2 Payment_Method,
  3 COUNT(*) AS Total_Transactions
  4 FROM Payment
  5 GROUP BY Payment_Method
  6 ORDER BY Total_Transactions DESC;
```

PAYMENT_METHOD	TOTAL_TRANSACTIONS
UPI	1
Debit Card	1
Credit Card	1

10. Calculate Amount Collected by Payment Mode

```
SQL> SELECT
  2     Payment_Method,
  3     SUM(Amount_Paid) AS Total_Amount_Collected
  4 FROM Payment
  5 WHERE Payment_Status = 'Successful'
  6 GROUP BY Payment_Method;
```

PAYMENT_METHOD	TOTAL_AMOUNT_COLLECTED
UPI	2098
Credit Card	1599
Debit Card	899

11. Payment Transaction Report

```
SQL> SELECT
  2     p.Payment_ID,
  3     p.Order_ID,
  4     p.Payment_Method,
  5     p.Payment_Date,
  6     p.Payment_Status,
  7     p.Amount_Paid
  8 FROM Payment p
  9 JOIN Orders o
 10     ON p.Order_ID = o.Order_ID
 11 ORDER BY p.Payment_Date;
```

PAYMENT_ID	ORDER_ID	PAYMENT_METHOD	PAYMENT_D
501	1001	UPI	29-SEP-26
Successful		2098	
503	1003	Debit Card	29-SEP-26
Successful		899	
502	1002	Credit Card	29-SEP-26
Successful		1599	